HW4 Reflection: Claude Code Workflow & TDD

Date: March 22, 2026

1. Comparing the Explore→Plan→Implement→Commit Workflow to Previous Approaches

Before using Claude Code, my AI development workflow was pretty fragmented. I usually kept switching between my IDE and a browser-based AI interface. If I ran into an issue or needed to build something new, I had to type out the problem, copy-paste my code, wait for the AI to give me an answer, and then try to fit that code back into my project. This back-and-forth process often caused bugs because the AI didn't know how my whole project was connected. On top of that, testing was usually just an afterthought that I did at the very end.

The **Explore→Plan→Implement→Commit** workflow inside Claude Code changed how I work. Having the AI right inside my terminal meant it had direct access to the actual files in my codebase. When we worked on the **Instructor Module Management API (Create/Edit/Delete)**, the difference was obvious right away:

The Explore Phase

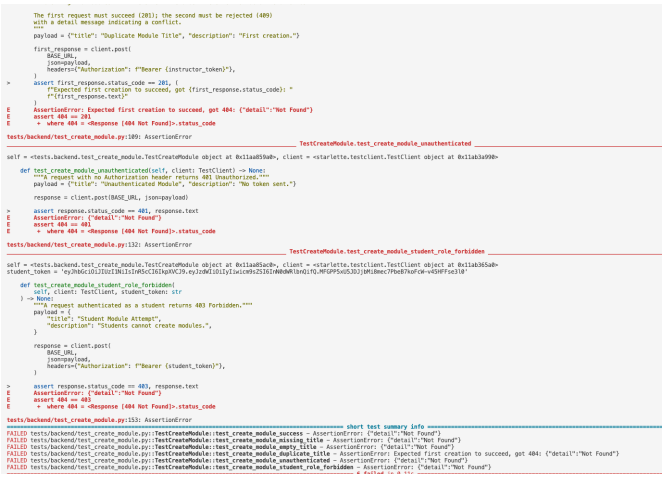
Instead of me having to explain how our code is set up, Claude used terminal commands like `Glob` and `Grep` to look at our FastAPI routes, Pydantic models, and Pytest configuration before writing any code. This made sure the new API features actually fit into our existing project and reused things like our current authentication code, rather than making up a new structure from scratch.

The Plan Phase

Instead of just asking the AI to "write the endpoint," we took a step back and made a plan. We wrote down exactly which edge cases we needed for the Create, Edit, and Delete routes. For example, we planned out the 200/201 success paths, the 422 errors for missing titles, the 409 errors for duplicate titles, and the 401/403 authorization checks. Planning this out beforehand saved us from the usual trial and error coding loops.

Implementation Driven by Testing (TDD)

This was the biggest change for me. We followed a strict Test-Driven Development (Red-Green-Refactor) loop. Before writing any actual business logic, Claude wrote out the Pytest files based on our plan (the RED phase). We ended up writing 17 tests across the three modules. We ran them in the terminal to watch them fail, which proved that the tests were working correctly.



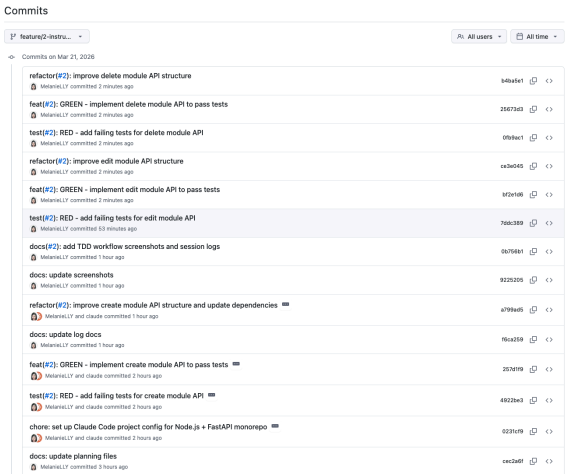
```
        The first request must succeed (201); the second must be rejected (409)
        with a detail message indicating a conflict.
        """
        payload = {"title": "Duplicate Module Title", "description": "First creation."}

        first_response = client.post(
            BASE_URL,
            json=payload,
            headers={"Authorization": f"Bearer {instructor_token}"},
        )
>       assert first_response.status_code == 201, (
            f"Expected first creation to succeed, got {first_response.status_code): "
            f"{first_response.text}"
        )
E       AssertionError: Expected first creation to succeed, got 404: {"detail":"Not Found"}
E       assert 404 == 201
E        +  where 404 = <Response [404 Not Found]>.status_code

tests/backend/test_create_module.py:189: AssertionError
_____ TestCreateModule.test_create_module_unauthenticated _____

self = <tests.backend.test_create_module.TestCreateModule object at 0x11aa8590a0>, client = <starlette.testclient.TestClient object at 0x11ab3a990>

    def test_create_module_unauthenticated(self, client: TestClient) -> None:
        """A request with no Authorization header returns 401 Unauthorized."""
        payload = {"title": "Unauthenticated Module", "description": "No token sent."}

        response = client.post(BASE_URL, json=payload)

>       assert response.status_code == 401, response.text
E       AssertionError: {"detail":"Not Found"}
E       assert 404 == 401
E        +  where 404 = <Response [404 Not Found]>.status_code

tests/backend/test_create_module.py:132: AssertionError
_____ TestCreateModule.test_create_module_student_role_forbidden _____

self = <tests.backend.test_create_module.TestCreateModule object at 0x11aa89ac0>, client = <starlette.testclient.TestClient object at 0x11ab365a0>
student_token = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyIiwicm9zZSI6IzNKbWNlbnQiTQ.MFGPP5xU5JOjbMi8mec7PbeB7koFcW-v45HFFse3l0'

    def test_create_module_student_role_forbidden(
        self, client: TestClient, student_token: str
    ) -> None:
        """A request authenticated as a student returns 403 Forbidden."""
        payload = {
            "title": "Student Module Attempt",
            "description": "Students cannot create modules.",
        }

        response = client.post(
            BASE_URL,
            json=payload,
            headers={"Authorization": f"Bearer {student_token}"},
        )

>       assert response.status_code == 403, response.text
E       AssertionError: {"detail":"Not Found"}
E       assert 404 == 403
E        +  where 404 = <Response [404 Not Found]>.status_code

tests/backend/test_create_module.py:153: AssertionError
=========================== short test summary info ============================
FAILED tests/backend/test_create_module.py::TestCreateModule::test_create_module_success - AssertionError: {"detail":"Not Found"}
FAILED tests/backend/test_create_module.py::TestCreateModule::test_create_module_missing_title - AssertionError: {"detail":"Not Found"}
FAILED tests/backend/test_create_module.py::TestCreateModule::test_create_module_empty_title - AssertionError: {"detail":"Not Found"}
FAILED tests/backend/test_create_module.py::TestCreateModule::test_create_module_duplicate_title - AssertionError: Expected first creation to succeed, got 404: {"detail":"Not Found"}
FAILED tests/backend/test_create_module.py::TestCreateModule::test_create_module_unauthenticated - AssertionError: {"detail":"Not Found"}
FAILED tests/backend/test_create_module.py::TestCreateModule::test_create_module_student_role_forbidden - AssertionError: {"detail":"Not Found"}
```

Caption: The RED phase, demonstrating failing tests for the Create Module API before any routing logic was implemented.

Only after confirming the failing tests did we write the minimum code needed to make them pass (the GREEN phase). Once Pytest showed a 100% pass rate, we went back to refactor the code (the REFACTOR phase) to make sure it followed PEP 8 style guidelines and had good docstrings. TDD basically forced us to prove our code worked instead of just running the server and hoping for the best.

Step-by-Step Commits

Because we developed in small, verified steps, our Git history turned out really clean. We made simple, step-by-step commits (like `test(#2): RED - add failing tests for create module API`, followed by `feat(#2): GREEN...`). This left us with a git log that actually tells the story of how we built the feature, which is a lot better than the massive, messy "initial commit" dumps I used to do.



Caption: A segment of our final Git log, clearly showing the Red-Green-Refactor commit history for the Instructor Module endpoints.

Overall, this loop broke my habit of constant context-switching and made testing the main focus of the project.

---

2. Context Management Strategies That Worked Best

For this assignment, we had to implement three different APIs, and running Pytest created a lot of terminal output. When using an AI agent in the terminal, managing the context window is super important so the AI doesn't get confused or slow down. Two specific strategies really helped us here:

Using `CLAUDE.md` as a Guide

Setting up a good `CLAUDE.md` file saved us a lot of time. By writing down our exact tech stack (Python 3.12, FastAPI, SQLAlchemy, Pytest) and our coding rules (PEP 8, docstrings), Claude knew exactly what to do from the start. When we ran `/init`, Claude even noticed some outdated Next.js and Prisma commands left over from an old setup and fixed them for us. Because this file was always loaded in the background, I never had to waste prompt tokens reminding the AI that we were building a FastAPI backend.

Clearing the Terminal with `/compact`

Since our TDD process involved running tests that threw long Python stack tracebacks when they failed, the working memory got full pretty fast. To keep the agent from getting slowed down by old error logs, the `/compact` command was a lifesaver.

We used `/compact` to throw away the long error logs while keeping a short summary of the progress we had made. This kept the agent focused. For example, when we finished the Create API and moved on to the Edit API, running `/compact` made sure Claude wasn't still thinking about the error messages from the previous API's RED phase. It gave us a clean slate for the next task without erasing the knowledge of what we just built.

Conclusion

Moving away from manually copying and pasting code to using the Explore→Plan→Implement→Commit workflow has made my coding process a lot more organized. By sticking to TDD and using context management commands like `/compact` and a proper `CLAUDE.md` file, we were able to finish the Instructor Module Management API with a clean codebase and a Git history that I'm actually proud to turn in.